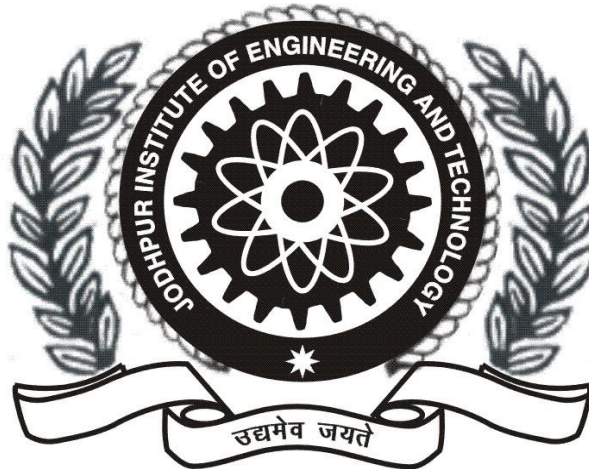


A PROJECT REPORT ON
LIBRARY MANAGEMENT SYSTEM

In partial fulfillment of

B.Tech II yr (Computer Science & Engg.)



Submitted by:

RAJVEER SINGH
CSE, II Year

Submitted To:

Prof. PARINITA KALLA
Assistant Professor

ACKNOWLEDGEMENT

I would like to acknowledge the contributions of the following people without whose help and guidance this project would not have been completed.

I respectfully thank **Prof. PARINITA KALLA**, for providing me an opportunity to do this project work and giving me all support and guidance, which made me complete the project up to very extent.

I am also thankful to **Prof Anju Jangid** HoD of Computer Science and Engineering Department, Jodhpur Institute of Engineering and Technology, for his/her constant encouragement, valuable suggestions and moral support and blessings.

Although it is not possible to name individually, I shall ever remain indebted to the faculty members of Jodhpur Institute of Engineering and Technology, for their persistent support and cooperation extended during his work.

This acknowledgement will remain incomplete if I fail to express our deep sense of obligation to my parents and God for their consistent blessings and encouragement.

Table of Contents

1. INTRODUCTION

2. TECHNOLOGY USED IN PROJECT

3. DETAILS OF PROJECT

1. Functions/Modules Details

2. Flow Chart

3. Project Code

4. Project Screenshots

4. APPLICATIONS

5. CONCLUSION AND FUTURE WORK

6. REFERENCES

1. INTRODUCTION

The <**Library Management System**> is a Java-based project designed to simplify and automate library operations. This application provides a structured way to manage books, issue them to students, and maintain accurate records of issued and returned books. Built using core Java features, along with a MySQL database for persistent data storage, it offers a user-friendly interface and robust functionality for handling daily library tasks.

Purpose

The purpose of this project is to provide an efficient and reliable system for managing a library's inventory, tracking book issues, and processing returns. The manual process of handling books in libraries can be time-consuming and prone to errors. By transitioning to an automated system, libraries can streamline their workflow, reduce human error, and enhance the overall experience for both librarians and users.

Key Features

- **Book Management:** The system allows librarians to add, view, and manage books. Each book has an associated ID, title, author, publisher, and publication year.
- **Issuing Books:** Students can borrow books, and the system records the issue date along with a return date, automatically calculated as 15 days after the issue date.
- **Returning Books:** The system enables the easy return of books, updating their status to available and removing them from the issued books record.
- **Viewing Records:** Users can view the list of available books and track issued books, including details such as student names and due dates.
- **JDBC:** Java Database Connectivity (JDBC) is used to connect the Java application to the MySQL database for executing queries.

2. Technology used in Project

The Library Management System (LMS) is developed using several key technologies that ensure its functionality and user-friendliness. Here's a clear overview of these technologies:

1.Java:

→The backbone of the application, providing a robust and platform-independent programming environment.

2.Swing:

→A user-friendly Java toolkit that creates an attractive graphical interface, allowing users to interact with the system effortlessly.

3.MySQL:

→A reliable database management system that stores all book and user data, ensuring quick access and secure data handling.

4.JDBC (Java Database Connectivity):

→This API bridges the application and the MySQL database, enabling smooth execution of SQL queries and managing connections effectively.

5.Exception Handling:

→Custom error management ensures that any issues are handled gracefully, enhancing the reliability of the application.

Modules/Functions

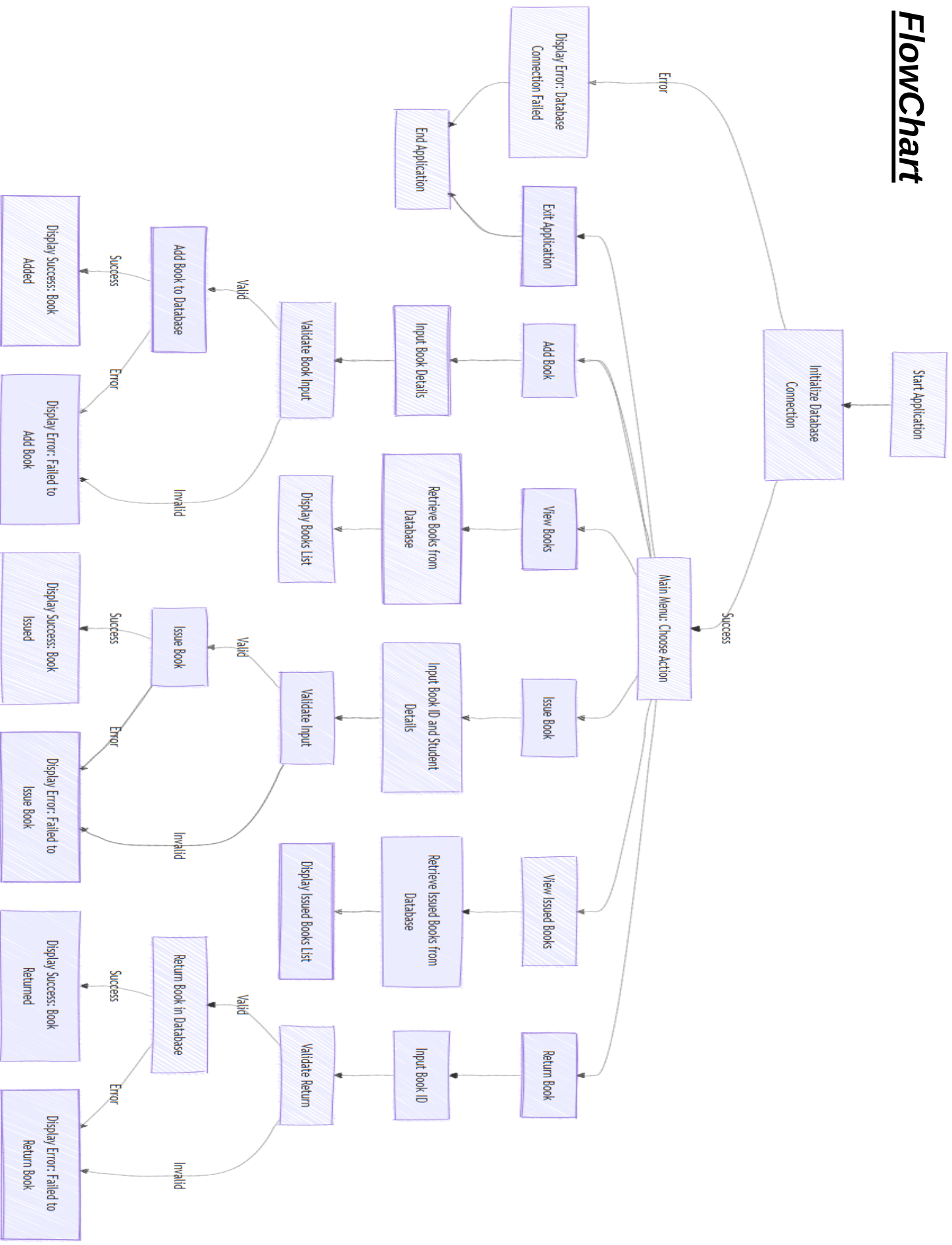
1.Database Connection: Manages the connection to the MySQL database.

2.Book Management:

- **Add Book:** Allows adding new books with validation.
- **View Books:** Displays a list of all books with their availability status.
- **Issue Book:** Issues a book to a student and sets the return date to 15 days from the issue date.
- **View Issued Books:** Displays all books that have been issued along with details of the student.
- **Return Book:** Handles returning of books, updating their status in the database.

3.User Interface: A Swing-based GUI that provides tabs for each functionality, enhancing user interaction.

FlowChart



SQL Command

Database:

Create database library_managenet;

Books Table:

```
CREATE TABLE books (  
    book_id INT PRIMARY KEY,  
    title VARCHAR(255),  
    author VARCHAR(255),  
    publisher VARCHAR(255),  
    year INT  
);
```

IssuedBooks Table:

```
CREATE TABLE issued_books (  
    issue_id INT AUTO_INCREMENT PRIMARY KEY,  
    book_id INT,  
    student_name VARCHAR(255),  
    reg_number VARCHAR(50),  
    issue_date DATE DEFAULT CURRENT_DATE,  
    return_date DATE DEFAULT (CURRENT_DATE + INTERVAL 15  
DAY),  
    FOREIGN KEY (book_id) REFERENCES books(book_id)  
);
```

Project Code:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.*;
import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Date;
// Custom exception for database operations
class DatabaseException extends Exception {
    public DatabaseException(String message) {
        super(message);}}
// Database connection manager
class DatabaseConnection {
    private Connection connection;
    public DatabaseConnection(String url, String user, String password) throws
DatabaseException {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        connection = DriverManager.getConnection(url, user, password);
        System.out.println("Database connected successfully!");
    } catch (ClassNotFoundException e) {
        throw new DatabaseException("MySQL Driver not found: " + e.getMessage());
    } catch (SQLException e) {
        throw new DatabaseException("Failed to connect to the database: " +
e.getMessage());}}
    public Connection getConnection() {
        return connection;}
    public void closeConnection() throws DatabaseException {
        if (connection != null) {
            try {
                connection.close();
                System.out.println("Database connection closed.");
            } catch (SQLException e) {
                throw new DatabaseException("Error while closing the connection: " +
e.getMessage());}}}}

// Book manager
class BookManager {
    private final Connection connection;
    private static final String ADD_BOOK_QUERY = "INSERT INTO books (id, title, author,
publisher, year, available) VALUES (?, ?, ?, ?, ?, TRUE)";
    private static final String VIEW_BOOKS_QUERY = "SELECT b.id, b.title, b.author,
b.publisher, b.year, (CASE WHEN ib.book_id IS NOT NULL THEN 'Issued' ELSE 'Available' END) AS
status FROM books b LEFT JOIN issued_books ib ON b.id = ib.book_id";
    private static final String ISSUE_BOOK_QUERY = "INSERT INTO issued_books (book_id,
student_name, registration_number, issue_date, return_date) VALUES (?, ?, ?, ?, ?)";
    private static final String VIEW_ISSUED_BOOKS_QUERY = "SELECT ib.id, ib.book_id, b.title,
ib.student_name, ib.registration_number, ib.issue_date, ib.return_date FROM issued_books ib
JOIN books b ON ib.book_id = b.id";
    public BookManager(DatabaseConnection db) throws DatabaseException {
        this.connection = db.getConnection();
        if (this.connection == null) {
            throw new DatabaseException("Failed to initialize BookManager: connection is
null.");}}
```

```

private boolean isValidBook(int id, String title, String author, String publisher, int
year) {
    return title != null && !title.isEmpty() && author != null && !author.isEmpty() &&
        publisher != null && !publisher.isEmpty() && year > 0 && id > 0;}
public boolean addBook(int id, String title, String author, String publisher, int year)
{
    if (!isValidBook(id, title, author, publisher, year)) {
        showErrorDialog("Invalid input, please check the book details.");
        return false;}
    try (PreparedStatement ps = connection.prepareStatement(ADD_BOOK_QUERY)) {
        ps.setInt(1, id);
        ps.setString(2, title);
        ps.setString(3, author);
        ps.setString(4, publisher);
        ps.setInt(5, year);
        int rowsAffected = ps.executeUpdate();
        if (rowsAffected > 0) {
            showInfoDialog("Book added successfully!");
            return true;
        } else {
            showErrorDialog("Failed to add the book.");
            return false;}
    } catch (SQLIntegrityConstraintViolationException e) {
        showErrorDialog("Error: Book with ID " + id + " already exists.");
        return false;
    } catch (SQLException e) {
        showErrorDialog("Error while adding book: " + e.getMessage());
        return false;}}
public DefaultTableModel getBooksTableModel() {
    DefaultTableModel model = new DefaultTableModel(new String[]{"ID", "Title",
"Author", "Publisher", "Year", "Status"}, 0);
    try (Statement stmt = connection.createStatement(); ResultSet rs =
stmt.executeQuery(VIEW_BOOKS_QUERY)) {
        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getInt("id"),
                rs.getString("title"),
                rs.getString("author"),
                rs.getString("publisher"),
                rs.getInt("year"),
                rs.getString("status")});}
    } catch (SQLException e) {
        showErrorDialog("Error while retrieving books: " + e.getMessage());}
    return model;}
public boolean issueBook(int bookId, String studentName, String registrationNumber) {
    String issueDate = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
    Calendar calendar = Calendar.getInstance();
    calendar.add(Calendar.DAY_OF_YEAR, 15);
    String returnDate = new SimpleDateFormat("yyyy-MM-dd").format(calendar.getTime());
    try (PreparedStatement ps = connection.prepareStatement(ISSUE_BOOK_QUERY)) {
        ps.setInt(1, bookId);
        ps.setString(2, studentName);
        ps.setString(3, registrationNumber);
        ps.setString(4, issueDate);
        ps.setString(5, returnDate);
    }
}

```

```

int rowsAffected = ps.executeUpdate();
    if (rowsAffected > 0) {
        showInfoDialog("Book issued successfully to " + studentName);
        return true;
    } else {
        showErrorDialog("Book cannot be issued (either does not exist or is already
issued).");
        return false;}
    } catch (SQLException e) {
        showErrorDialog("Error while issuing book: " + e.getMessage());
        return false;}}
    public DefaultTableModel getIssuedBooksTableModel() {
        DefaultTableModel model = new DefaultTableModel(new String[]{"ID", "Book ID",
"Title", "Student Name", "Reg No", "Issue Date", "Return Date"}, 0);
        try (Statement stmt = connection.createStatement(); ResultSet rs =
stmt.executeQuery(VIEW_ISSUED_BOOKS_QUERY)) {
            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getInt("id"),
                    rs.getInt("book_id"),
                    rs.getString("title"),
                    rs.getString("student_name"),
                    rs.getString("registration_number"),
                    rs.getString("issue_date"),
                    rs.getString("return_date")});}
        } catch (SQLException e) {
            showErrorDialog("Error while retrieving issued books: " + e.getMessage());}
        return model;}
    public boolean returnBook(int bookId) {
        try (PreparedStatement psUpdate = connection.prepareStatement("UPDATE books SET
available = TRUE WHERE id = ? AND available = FALSE");
            PreparedStatement psDelete = connection.prepareStatement("DELETE FROM
issued_books WHERE book_id = ?")) {
            psDelete.setInt(1, bookId);
            int rowsAffected = psDelete.executeUpdate();
            if (rowsAffected > 0) {
                psDelete.setInt(1, bookId);
                psUpdate.executeUpdate();
                showInfoDialog("Book returned successfully!");
                return true;
            } else {
                showErrorDialog("Book cannot be returned (either does not exist or was not
issued).");
                return false;}
        } catch (SQLException e) {
            showInfoDialog("Book returned successfully!");
            return false;}}
    private void showErrorDialog(String message) {
        JOptionPane.showMessageDialog(null, message, "Error", JOptionPane.ERROR_MESSAGE);}
    private void showInfoDialog(String message){
        JOptionPane.showMessageDialog(null, message, "Information",
JOptionPane.INFORMATION_MESSAGE);}

```

```

public class LibraryManagement extends JFrame {
    private DatabaseConnection db;
    private BookManager bookManager;
    private JTable booksTable;
    private JTable issuedBooksTable;
    private DefaultTableModel booksTableModel;
    private DefaultTableModel issuedBooksTableModel;
    public LibraryManagement() {
        try {
            db = new DatabaseConnection("jdbc:mysql://localhost:3306/library_management",
"root", "rks@2552");
            bookManager = new BookManager(db);
        } catch (DatabaseException e) {
            JOptionPane.showMessageDialog(null, e.getMessage(), "Initialization Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        setTitle("Library Management System");
        setSize(800, 600);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());
        JTabbedPane tabbedPane = new JTabbedPane();
        JPanel addBookPanel = createAddBookPanel();
        tabbedPane.addTab("Add Book", addBookPanel);
        JPanel viewBooksPanel = createViewBooksPanel();
        tabbedPane.addTab("View Books", viewBooksPanel);
        JPanel issueBookPanel = createIssueBookPanel();
        tabbedPane.addTab("Issue Book", issueBookPanel);
        JPanel viewIssuedBooksPanel = createViewIssuedBooksPanel();
        tabbedPane.addTab("View Issued Books", viewIssuedBooksPanel);
        JPanel returnBookPanel = createReturnBookPanel();
        tabbedPane.addTab("Return Book", returnBookPanel); // Add Return Book tab
        add(tabbedPane, BorderLayout.CENTER);
        setVisible(true);
    }
    private JPanel createAddBookPanel() {
        JPanel panel = new JPanel(new GridLayout(6, 2));
        JLabel idLabel = new JLabel("Book ID:");
        JTextField idField = new JTextField();
        JLabel titleLabel = new JLabel("Title:");
        JTextField titleField = new JTextField();
        JLabel authorLabel = new JLabel("Author:");
        JTextField authorField = new JTextField();
        JLabel publisherLabel = new JLabel("Publisher:");
        JTextField publisherField = new JTextField();
        JLabel yearLabel = new JLabel("Year:");
        JTextField yearField = new JTextField();
        JButton addButton = new JButton("Add Book");
        addButton.addActionListener(e -> {
            int id = Integer.parseInt(idField.getText());
            String title = titleField.getText();
            String author = authorField.getText();
            String publisher = publisherField.getText();
            int year = Integer.parseInt(yearField.getText());
            if (bookManager.addBook(id, title, author, publisher, year)) {
                refreshBooksTable();
            }
        });
    }
}

```

```

panel.add(idLabel);
panel.add(idField);
panel.add(titleLabel);
panel.add(titleField);
panel.add(authorLabel);
panel.add(authorField);
panel.add(publisherLabel);
panel.add(publisherField);
panel.add(yearLabel);
panel.add(yearField);
panel.add(new JLabel());
panel.add(addButton);
return panel;}

private JPanel createViewBooksPanel() {
    JPanel panel = new JPanel(new BorderLayout());
    booksTableModel = bookManager.getBooksTableModel();
    booksTable = new JTable(booksTableModel);
    JScrollPane scrollPane = new JScrollPane(booksTable);
    panel.add(scrollPane, BorderLayout.CENTER);
    JButton refreshButton = new JButton("Refresh");
    refreshButton.addActionListener(e -> refreshBooksTable());
    panel.add(refreshButton, BorderLayout.SOUTH); // Add refresh button to manually
refresh the view
    return panel;}

private void refreshBooksTable() {
    booksTableModel.setRowCount(0); // Clear existing data
    DefaultTableModel newModel = bookManager.getBooksTableModel();
    for (int i = 0; i < newModel.getRowCount(); i++) {
        booksTableModel.addRow(new Object[]{
            newModel.getValueAt(i, 0),
            newModel.getValueAt(i, 1),
            newModel.getValueAt(i, 2),
            newModel.getValueAt(i, 3),
            newModel.getValueAt(i, 4),
            newModel.getValueAt(i, 5)}});
    }

private JPanel createIssueBookPanel() {
    JPanel panel = new JPanel(new GridLayout(4, 2));
    JLabel bookIdLabel = new JLabel("Book ID:");
    JTextField bookIdField = new JTextField();
    JLabel studentNameLabel = new JLabel("Student Name:");
    JTextField studentNameField = new JTextField();
    JLabel registrationNumberLabel = new JLabel("Reg. Number:");
    JTextField registrationNumberField = new JTextField();
    JButton issueButton = new JButton("Issue Book");
    issueButton.addActionListener(e -> {
        int bookId = Integer.parseInt(bookIdField.getText());
        String studentName = studentNameField.getText();
        String registrationNumber = registrationNumberField.getText();
        if (bookManager.issueBook(bookId, studentName, registrationNumber)) {
            refreshBooksTable(); // Refresh books table after issuing
            refreshIssuedBooksTable();});
    panel.add(bookIdLabel);panel.add(bookIdField);
    panel.add(studentNameLabel);panel.add(studentNameField);
    panel.add(registrationNumberLabel);panel.add(registrationNumberField);
    panel.add(new JLabel()); // Empty cell
    panel.add(issueButton);
    return panel;}

```

```

private JPanel createViewIssuedBooksPanel() {
    JPanel panel = new JPanel(new BorderLayout());
    issuedBooksTableModel = bookManager.getIssuedBooksTableModel(); // Initialize the
issued books table model
    issuedBooksTable = new JTable(issuedBooksTableModel);
    JScrollPane scrollPane = new JScrollPane(issuedBooksTable);
    panel.add(scrollPane, BorderLayout.CENTER);
    // Add a refresh button to update the table
    JButton refreshButton = new JButton("Refresh");
    refreshButton.addActionListener(e -> refreshIssuedBooksTable()); // Refresh issued
books table on button click
    panel.add(refreshButton, BorderLayout.SOUTH); // Add the refresh button at the
bottom of the panel
    return panel;}

private void refreshIssuedBooksTable() {
    issuedBooksTableModel.setRowCount(0); // Clear existing data
    DefaultTableModel newModel = bookManager.getIssuedBooksTableModel(); // Fetch fresh
data
    for (int i = 0; i < newModel.getRowCount(); i++) {
        issuedBooksTableModel.addRow(new Object[] {
            newModel.getValueAt(i, 0),
            newModel.getValueAt(i, 1),
            newModel.getValueAt(i, 2),
            newModel.getValueAt(i, 3),
            newModel.getValueAt(i, 4),
            newModel.getValueAt(i, 5),
            newModel.getValueAt(i, 6)}));}

private JPanel createReturnBookPanel() {
    JPanel panel = new JPanel(new GridLayout(2, 2));
    JLabel bookIdLabel = new JLabel("Book ID:");
    JTextField bookIdField = new JTextField();
    JButton returnButton = new JButton("Return Book");
    returnButton.addActionListener(e -> {
        int bookId = Integer.parseInt(bookIdField.getText());
        if (bookManager.returnBook(bookId)) {
            refreshBooksTable(); // Refresh books table after returning
            refreshIssuedBooksTable();});}

    panel.add(bookIdLabel);
    panel.add(bookIdField);
    panel.add(new JLabel()); // Empty cell
    panel.add(returnButton);
    return panel;}

public static void main(String[] args) {
    SwingUtilities.invokeLater(LibraryManagement::new);}}

```

Project Screenshots

Library Management System

Add Book View Books Issue Book View Issued Books Return Book

Book ID: 77778888

Title: JavaScript: The Good Parts

Author:

Publisher: O'Reilly Media

Year: 2008

Add Book

Information

Book added successfully!

OK

Library Management System

Add Book View Books Issue Book View Issued Books Return Book

ID	Title	Author	Publisher	Year	Status
981245	The Clean Architecture: A Craftsman's Guide to	Robert C. Martin	Prentice Hall	2017	Issued
9876543	Ruby on Rails Tutorial	Michael Hartl	Addison-Wesley	2016	Available
13426789	Effective Java	Joshua Bloch	Addison-Wesley	2001	Available
31415926	Learning Python	Mark Lutz	O'Reilly Media	1999	Available
41818926	Code: The Hidden Language of Computer Hardw	Charles Petzold	Microsoft Press	1999	Available
55566666	Data Science from Scratch	Joel Grus	O'Reilly Media	2019	Issued
56231498	Refactoring: Improving the Design of Existing Co	Martin Fowler	Addison-Wesley	1999	Available
67890123	Learning SQL	Alan Beaulieu	O'Reilly Media	2009	Issued
75649032	Automate the Boring Stuff with Python	Al Sweigart	No Starch Press	2015	Available
77778888	JavaScript: The Good Parts	Douglas Crockford	O'Reilly Media	2008	Available
87129034	The Phoenix Project: A Novel About IT, DevOps,	Gene Kim, Kevin Behr, George Spafford	IT Revolution Press	2013	Available
98124567	The Clean Architecture: A Craftsman's Guide to	Robert C. Martin	Prentice Hall	2017	Available

Refresh

Library Management System

Add Book

View Books

Issue Book

View Issued Books

Return Book

ID	Book ID	Title	Student Name	Reg No	Issue Date	Return Date
26	981245	The Clean Architecture: A Craftsman's G	Rajveer singh	JIET/CS/23/082	2024-10-25	2024-11-09
27	67890123	Learning SQL	Rajat singharya	JIET/CS/23/032	2024-10-25	2024-11-09

Refresh

Library Management System

Add Book

View Books

Issue Book

View Issued Books

Return Book

Book ID:

55566666

Information

Book returned successfully!

OK

Return Book

4. APPLICATIONS

- The **Library Management System (LMS)** simplifies library operations, enhances accuracy, and improves user experience.

Key Applications:

- **Efficient Book Management:**
Maintains organized book inventory with easy updates and retrieval.
- **Automated Issuance and Return Processing:**
Tracks issued books and return dates automatically, ensuring smooth circulation.
- **Enhanced Data Security:**
Enforces unique book identifiers and secure data storage using MySQL.
- **Real-Time Availability Check:**
Allows staff to instantly check book availability for users.
- **User-Friendly Interface:**
Intuitive design for easy navigation, requiring minimal training.

Practical Uses:

- School, college, and university libraries
- Public and community libraries
- Private institution libraries
- Organizations needing resource tracking

5. FUTURE WORKS AND CONCLUSIONS

Future Works

- The current Library Management System lays a solid foundation, but there are several areas where future improvements can be made:
- **Advanced Search Features:** Integrating more advanced search filters like genre, keywords, and language to improve user experience when searching for books.
- **Automated Notifications:** Implementing email or SMS notifications for users to remind them of due dates and overdue returns.
- **Online Book Reservation:** Allowing users to reserve books online, which can then be picked up at the library.
- **Mobile Application:** Developing a mobile app version for easier access and management of library resources from smartphones.
- **Analytics and Reporting:** Adding advanced analytics to generate detailed reports about book trends, popular authors, and library usage patterns for better decision-making.

Conclusion

- The Library Management System has successfully automated key library functions, including book addition, issuing, and returns, ensuring efficient, fast, and error-free management of library resources. By eliminating manual operations, it not only saves time but also improves accuracy and overall productivity.
- With potential future enhancements like advanced search, notifications, and mobile access, the system has the capability to evolve into a more comprehensive and user-friendly tool for modern library management. Overall, this project demonstrates how technology can streamline traditionally complex processes, making libraries more accessible and manageable.

6. REFERENCES

- **JDBC Documentation:** Oracle. (n.d.). *JDBC API Reference and Documentation*. Retrieved from <https://docs.oracle.com/javase/8/docs/api/java/sql/package-summary.html>
- **MySQL Documentation:** MySQL. (n.d.). *MySQL 8.0 Reference Manual*. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>
- **Swing Tutorial:** Oracle. (n.d.). *Creating a GUI with Swing*. Retrieved from <https://docs.oracle.com/javase/tutorial/uiswing/>
- **Java SE Development Kit (JDK):** Oracle. (n.d.). *Java SE 8 Platform*. Retrieved from <https://www.oracle.com/java/technologies/javase/javase-jdk8-downloads.html>
- **Design Patterns for Java:** Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- **Effective Java:** Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
- **Database Management Systems:** Ramakrishnan, R., & Gehrke, J. (2002). *Database Management Systems* (3rd ed.). McGraw-Hill Education.
- **Library Management Software Principles:** Laudon, K. C., & Laudon, J. P. (2021). *Management Information Systems: Managing the Digital Firm* (16th ed.). Pearson.